

MODUL PRAKTIKUM REKAYASA PERANGKAT LUNAK

Pertemuan 12 & 13

GitHub, Pemeliharaan Perangkat Lunak, dan Arsitektur Perangkat Lunak

Dosen Pengampu:

M. Amran Hakim Siregar, S.Tr.Kom., M.Kom.

Daftar Isi

1. Tujuan Praktikum
2. Persiapan Sebelum Praktikum
3. Bagian A — Pertemuan 12: GitHub & Pemeliharaan Perangkat Lunak
4. Bagian B — Pertemuan 13: Dokumentasi Arsitektur Perangkat Lunak
5. Struktur Akhir Repository
6. Rubrik Penilaian
7. Pengumpulan Tugas
8. Troubleshooting Umum

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Membuat dan mengonfigurasi akun GitHub serta repository baru.
- Melakukan instalasi dan konfigurasi Git di komputer lokal.
- Melakukan operasi dasar Git: init, add, commit, push, dan pull.
- Memahami konsep Software Configuration Management (SCM) dan peran Git di dalamnya.
- Menyusun rencana pemeliharaan perangkat lunak (maintenance plan) yang terklasifikasi menjadi Corrective, Adaptive, Perfective, dan Preventive.
- Merancang dan mendokumentasikan arsitektur perangkat lunak sederhana, termasuk quality attributes, logical view, dan physical view.
- Mempublikasikan seluruh hasil kerja secara rapi di repository GitHub pribadi.

2. Persiapan Sebelum Praktikum

Pastikan hal-hal berikut sudah siap sebelum praktikum dimulai agar tidak menghambat proses pengerjaan:

- Koneksi internet yang stabil.
- Browser (Chrome/Firefox/Edge) untuk mengakses github.com.
- Git sudah terpasang di laptop/komputer. Cek dengan perintah: `git --version`
- Text editor (disarankan Visual Studio Code) untuk menulis file Markdown (.md).
- Akun email aktif untuk pendaftaran GitHub (gunakan email pribadi/kampus yang masih aktif).

Mengecek/Memasang Git

Jika perintah `git --version` menampilkan error "command not found", unduh dan pasang Git terlebih dahulu dari <https://git-scm.com/downloads> sesuai sistem operasi yang digunakan (Windows/Mac/Linux).

Catatan: Setelah Git terpasang, lakukan konfigurasi nama dan email satu kali saja (akan dipakai di setiap commit):

```
git config --global user.name "Nama Lengkap Anda"
git config --global user.email "email_anda@contoh.com"
```

3. Bagian A — Pertemuan 12: GitHub & Pemeliharaan Perangkat Lunak

3.1 Konsep Singkat: SCM dan Git

Software Configuration Management (SCM) adalah praktik untuk mengelola dan mengontrol perubahan pada perangkat lunak selama siklus hidupnya, mencakup version control, change tracking, dan kolaborasi tim. Git adalah salah satu tool SCM (version control system) yang paling banyak digunakan, sedangkan GitHub adalah platform hosting untuk repository Git secara online.

3.2 Langkah Kerja

Langkah 1 — Membuat Akun GitHub

1. Buka <https://github.com> lalu klik Sign Up.
2. Isi username, email, dan password, kemudian verifikasi akun melalui email.
3. Setelah berhasil login, lengkapi profil dasar (foto profil dan nama, opsional).

Langkah 2 — Membuat Repository Baru

1. Klik tombol + di pojok kanan atas, pilih New repository.
2. Beri nama repository dengan format: RPL-NIM-NAMA (contoh: RPL-2210512345-BudiSantoso).

3. Pilih Public, lalu centang Add a README file.
4. Klik Create repository.

Langkah 3 — Clone Repository ke Komputer Lokal

Salin URL repository (tombol hijau Code → HTTPS), lalu jalankan di terminal:

```
git clone https://github.com/username-anda/RPL-NIM-NAMA.git
cd RPL-NIM-NAMA
```

Langkah 4 — Melengkapi README.md

Buka file README.md dengan text editor, lalu lengkapi minimal informasi berikut:

- Judul repository dan nama mahasiswa/NIM.
- Deskripsi singkat tujuan repository (untuk tugas Pertemuan 12 & 13 RPL).
- Daftar isi folder yang ada di repository.

Langkah 5 — Membuat Folder dan File Maintenance Plan

Buat folder bernama maintenance, lalu di dalamnya buat file maintenance-plan.md.

Isi file tersebut dengan minimal 10 kebutuhan pemeliharaan perangkat lunak, dikelompokkan ke dalam 4 kategori berikut:

Kategori	Penjelasan
Corrective	Perbaikan terhadap bug atau kesalahan yang ditemukan setelah sistem digunakan (contoh: memperbaiki error saat login gagal).
Adaptive	Penyesuaian sistem terhadap perubahan lingkungan, seperti versi OS baru, perubahan regulasi, atau API pihak ketiga yang berubah.
Perfective	Peningkatan fitur atau performa sistem yang sudah berjalan baik, misalnya menambah fitur pencarian atau mempercepat loading halaman.
Preventive	Tindakan pencegahan agar masalah tidak muncul di masa depan, seperti refactoring kode atau pembaruan dokumentasi.

Contoh format penulisan di dalam maintenance-plan.md:

```
# Maintenance Plan

## Corrective
1. Memperbaiki bug validasi form pendaftaran yang menerima input kosong.
2. Memperbaiki kesalahan perhitungan total harga pada halaman checkout.

## Adaptive
3. Menyesuaikan API pembayaran karena penyedia mengubah versi endpoint.
...

## Perfective
...
```

```
## Preventive
...
```

Catatan: Setiap kategori sebaiknya memiliki minimal 2-3 poin agar total keseluruhan mencapai 10 kebutuhan pemeliharaan.

Langkah 6 — Commit dan Push Hasil Pekerjaan

Setelah semua file selesai diedit dan disimpan, jalankan perintah berikut secara berurutan dari terminal (pastikan posisi terminal berada di dalam folder repository):

```
git add .
git commit -m "Add maintenance plan"
git push origin main
```

Penting: Jika diminta login, gunakan username GitHub dan Personal Access Token (bukan password akun) sebagai kredensial. Token dapat dibuat di Settings > Developer settings > Personal access tokens.

4. Bagian B — Pertemuan 13: Dokumentasi Arsitektur Perangkat Lunak

4.1 Konsep Singkat: Arsitektur Perangkat Lunak

Arsitektur perangkat lunak adalah struktur dasar suatu sistem yang mencakup komponen-komponen, hubungan antarkomponen, serta prinsip yang mengatur desain dan evolusinya. Dokumentasi arsitektur membantu tim memahami bagaimana sistem bekerja secara keseluruhan sebelum masuk ke detail implementasi.

4.2 Langkah Kerja

Langkah 1 — Memilih Sebuah Sistem

Pilih satu sistem perangkat lunak sederhana sebagai studi kasus. Sistem dapat berupa aplikasi yang sudah ada (contoh: aplikasi e-commerce sederhana, sistem akademik kampus, aplikasi pemesanan tiket) atau ide aplikasi yang ingin dirancang sendiri.

Langkah 2 — Membuat Folder Architecture

Di dalam repository yang sama, buat folder bernama architecture. Di dalam folder ini, buat 5 file Markdown berikut:

Nama File	Isi
deskripsi-sistem.md	Penjelasan singkat tentang sistem yang dipilih: nama, tujuan, pengguna utama, dan fitur-fitur utamanya.
quality-attributes.md	Daftar atribut kualitas yang menjadi prioritas sistem (contoh: performance, security, scalability, usability) beserta alasan singkat.

Nama File	Isi
arsitektur.md	Penjelasan pola/gaya arsitektur yang digunakan (contoh: layered architecture, microservices, client-server) beserta alasan pemilihannya.
logical-view.md	Gambaran komponen/modul logis sistem dan hubungan antarmodul (boleh disertai diagram sederhana, misalnya dibuat dengan draw.io lalu screenshot, atau diagram teks).
physical-view.md	Gambaran infrastruktur fisik/deployment sistem: server, database, jaringan, dan bagaimana komponen-komponen tersebut saling terhubung.

Langkah 3 — Commit dan Push

```
git add .
git commit -m "Add software architecture documentation"
git push origin main
```

Catatan: Jika ingin menyertakan gambar/diagram, simpan gambar di dalam folder `architecture/images/` lalu sisipkan di file Markdown dengan format: `![Nama Diagram](images/nama-file.png)`

5. Struktur Akhir Repository

Setelah Pertemuan 12 dan 13 selesai dikerjakan, struktur repository pada GitHub harus terlihat seperti berikut:

```
RPL-NIM-NAMA/
├── README.md
├── maintenance/
│   └── maintenance-plan.md
├── architecture/
│   ├── deskripsi-sistem.md
│   ├── quality-attributes.md
│   ├── arsitektur.md
│   ├── logical-view.md
│   └── physical-view.md
```

6. Rubrik Penilaian

Komponen Penilaian	Bobot
Akun GitHub aktif & repository sesuai format penamaan	10

Komponen Penilaian	Bobot
Repository (visibility, kelengkapan, kerapian pengaturan)	10
README.md (judul, deskripsi, daftar isi)	10
Maintenance Plan (minimal 10 poin, 4 kategori lengkap)	20
Struktur folder sesuai ketentuan	10
Quality Attributes (relevan dan beralasan)	10
Dokumentasi Arsitektur (5 file lengkap & jelas)	15
Commit History (pesan commit jelas & bertahap)	10
Kerapian keseluruhan repository	5

Total Nilai: 100

7. Pengumpulan Tugas

1. Pastikan seluruh file sudah ter-push ke GitHub sebelum batas waktu pengumpulan (Lihat deadline pada portal akademik RPL).
2. Salin tautan (URL) repository GitHub Anda ke Tugas.
3. Kumpulkan tautan tersebut melalui platform Portal Akademik RPL.
4. Pastikan repository bersifat Public agar dapat diakses dan dinilai.

8. Troubleshooting Umum

Masalah	Solusi
git: command not found	Pasang Git dari git-scm.com, lalu restart terminal.
Permission denied / authentication failed saat push	Gunakan Personal Access Token sebagai pengganti password, atau atur autentikasi SSH.
Repository sudah ada isinya saat clone gagal ("fatal: destination path already exists")	Hapus folder lokal yang sudah ada, atau clone ke folder dengan nama berbeda.
Lupa menambahkan file sebelum commit	Jalankan kembali git add . sebelum git commit.
Perubahan tidak muncul di GitHub setelah push	Pastikan berada di branch yang benar (git branch) dan push ke branch yang sama dengan yang ditampilkan di GitHub (biasanya main).
